

Jaxtyping Shape Contracts in GiGL

1 What changed

Tensor annotations at stable GiGL boundaries now state dtype, rank, and axis relationships. The annotations remain part of the production API, but runtime checking is enabled only by the unit, integration, and end-to-end test launchers.

- `jaxtyping` is a runtime dependency because annotations are imported with GiGL modules.
- `typeguard>=2,<3` is a test dependency and is not installed for production execution.
- The launchers install one Jaxtyping import hook for `gigl` and `examples` before test discovery.
- A custom adapter gives Typeguard only annotations that contain a Jaxtyping array type. Existing static-only annotations are not turned into runtime policy.

2 Contract syntax used in GiGL

Axis names are local to one checked call. Reusing a name across parameters and the return value requires the sizes to match.

Contract	Meaning
<code>Float[Tensor, "queries embedding_dim"]</code>	Floating tensor with rank 2 and named axes.
<code>Int64[Tensor, "nodes"]</code>	One-dimensional <code>torch.int64</code> tensor.
<code>Float[Tensor, "... feature_dim"]</code>	Any number of leading axes followed by <code>feature_dim</code> .
<code>Float[Tensor, "_nodes output_dim"]</code>	<code>_nodes</code> is descriptive but does not bind to another annotation.
<code>Float[Tensor, "#batch features"]</code>	<code>batch</code> may match its bound size or be 1 for broadcasting.
<code>Float32[Tensor, "... {output_dim}"]</code>	Final size is read from a runtime argument or instance field.
<code>Float[Tensor, ""]</code>	Scalar tensor.

Use exact dtypes only where the interface guarantees them. Model and loss boundaries use `Float` when mixed precision is valid; ID tensors and packed features use `Int64` and `UInt8`.

3 Examples from current GiGL APIs

The decoder contract ties the input embedding widths together and carries the input counts into the score matrix:

```
def decode(
    query_embeddings: Float[Tensor, "queries embedding_dim"],
    candidate_embeddings: Float[Tensor, "candidates embedding_dim"],
) -> Float[Tensor, "queries candidates"]: ...
```

Heterogeneous outputs do not require every node type to contain the same number of nodes:

```
def forward(...) -> dict[
    NodeType, Float[Tensor, "_nodes embedding_dim"]
]: ...
```

Contracts also apply inside containers. This catches a malformed layer output, not merely a malformed list object:

```
def forward(
    self, xs: list[Float[Tensor, "nodes hidden_dim"]]
) -> Float[Tensor, "nodes output_dim"]: ...
```

4 Runtime enforcement path

Test entry points	tests/unit/main.py, tests/integration/main.py, and tests/e2e_tests/e2e_test.py call <code>install_runtime_typechecking()</code> before importing GiGL modules.
Import scope	<code>install_import_hook()</code> instruments subsequently imported functions and dataclasses in <code>gigl</code> and examples.
Annotation filter	<code>shape_contract_typechecker()</code> returns uncontracted functions unchanged. For contracted functions, it creates a private function copy containing only shape-bearing annotations.
Runtime check	Typeguard checks arguments before the call and the return value afterward. Jaxtyping binds named axes for that call.
Failure	A dtype, rank, or axis mismatch raises <code>jaxtyping.TypeCheckError</code> ; an uncaught error fails the test command.

The private function copy matters because Typeguard reads `__annotations__` at call time. The adapter restores the complete public annotation mapping on the wrapped function, so runtime filtering does not change the API seen by introspection or static tooling.

5 Why Typeguard 2.x instead of Beartype

Both libraries integrate with Jaxtyping and both were viable. GiGL uses Typeguard for deterministic container coverage.

Criterion	Typeguard 2.x	Beartype
Container checking	Exhaustively checks values reached through annotated containers.	Samples container contents to keep checking near constant time.
GiGL impact	Deterministically checks contracts such as <code>list[float[...]]</code> , <code>dict[NodeType, float[...]]</code> , and nested loader inputs.	A malformed tensor can be missed when its container element is not sampled.
Runtime cost	Higher for large containers, accepted because checking runs only in tests.	Lower and more suitable when runtime overhead is the primary constraint.
Jaxtyping support	Supported, with the 2.x series required because Jaxtyping documents known issues in Typeguard 3 and 4.	Supported without the Typeguard version constraint.

The custom annotation filter is separate from this choice. It prevents either checker from treating all existing GiGL annotations as runtime contracts. TypeVars bounded by non-runtime-checkable Protocols and unresolved forward references can otherwise fail during `isinstance` evaluation.

6 Enforcement boundaries

- Checks run only for calls reached by tests through the standard GiGL test launchers. Commands that bypass those entry points do not install the hook.
- The hook affects modules imported after installation; this is why installation precedes test discovery and GiGL imports.
- Only annotations containing a Jaxtyping array type are checked. This is not repository-wide runtime validation of ordinary Python annotations.
- Axis bindings apply within one checked function call. The same name in separate calls does not create shared state.
- A leading underscore makes an axis independent. Use it for heterogeneous dictionary values whose node counts legitimately differ by key.

References: [Jaxtyping array annotations](#); [runtime type checking](#); [Typeguard and Beartype checking behavior](#).